Detailed Task Distribution & Integration Guide

SIH Internal Round Prototype | PS 34 — Legal Metrology Automation

For every member: **What to Build** (specific, step-by-step), **Connects With** (what you need from others, and who's waiting on your output), and **Demo Deliverable**. The Integration Guide at the end shows exactly how all six pieces get wired into one working system.

MAIN TASKS

1. Backend & Database Architect

NestJS API Gateway, PostgreSQL schema, workflow/state logic

What to Build

1. Database schema (PostgreSQL) — these tables, minimum:

- **users** (id, role, name, phone, email, keycloak_id)
- **businesses** (id, owner_user_id, name, address, geo_lat, geo_lng, gstin, turnover_band)
- **complaints** (id, citizen_id, business_id [nullable], retailer_name_text, retailer_address_text, category, description, photo_urls[], invoice_url, status)
- **inspections** (id, inspector_id, business_id, complaint_id [nullable], visit_date, status)
- **ocr_results** (id, inspection_id/self_check_id, image_urls[], extracted_fields JSON, violations JSON, offence_tier)
- **self_check_reports** (id, business_id, image_urls[], extracted_fields JSON, violations JSON) — its own table, never joined into inspector/controller queries
- **notices** (id, case_id, type, content_text, section_refs, status, issued_date, deadline_date, pdf_url, signature_url)
- **offence_records** (id, product_name_normalized, manufacturer_normalized, business_id, notice_id, offence_number, date)
- **disputes** (id, notice_id, business_id, comment, status)
- **payments** (id, notice_id, business_id, amount, razorpay_order_id, status)
- **supply_chain_links** (id, source_business_id, named_business_id, inspection_id, status)

2. REST API endpoints (NestJS):

- `POST /complaints` — citizen files a complaint
- `GET /complaints/:id/status` — status tracker
- `GET /complaints?region=` — inspector/controller queue
- `POST /businesses` — register a business
- `POST /inspections` — start an inspection
- `POST /inspections/:id/ocr-result` — receives OCR output
- `GET /offence-history?product=&manufacturer=` — offence tier check
- `POST /notices` — create/store a notice (calls the AI service's NLP endpoint internally)
- `PATCH /notices/:id` — status changes (acknowledged, disputed, approved, rejected, paid)
- `POST /self-check` — business submits packaging photos (private path)
- `POST /disputes`
- `POST /payments/initiate`, `POST /payments/webhook` (Razorpay callback)
- `GET /controller/dashboard/stats`
- `POST /supply-chain-links`, `PATCH /supply-chain-links/:id/assign`

3. **Keycloak setup** — 4 roles (Citizen, Business, Inspector, Controller), JWT validation middleware, route guards per role.
4. **Case workflow logic** — a CaseWorkflowService enforcing valid status transitions: Received → Assigned → Inspected → Notice Issued → Compounded/Prosecution → Resolved. Reject any transition that skips a stage.
5. **Deadline & offence logic** — days_remaining = deadline_date minus today, computed per notice type using the day-counts from the Rulebook; offence-tier lookup either done here or delegated to the AI service's fuzzy-match function (agree this with Member 2 directly).

Connects With

Needs from Member 5 (Rulebook):
penalty amounts and deadline-day counts per notice type — needed by end of Week 1 to finalize the schema, even in draft form.

Needs from Member 2 (AI/OCR):
the exact JSON shape of the OCR result and notice-generation response, agreed on Day 1–2, so the ocr_results and notices tables match what's actually returned.

Provides to Member 2:
the API contract for how their service gets called (base URL, expected input format for images/case data).

Provides to Member 3 & 4 (Frontend):
a Postman collection / Swagger docs with sample requests and responses for every endpoint above — this unblocks them to start building screens before the backend is fully live.

Provides to Member 6:
the /payments/webhook endpoint ready to receive and test Razorpay callbacks.

Demo Deliverable

Swagger/Postman collection showing every endpoint working with sample data — a complaint can be filed, an inspection opened, a notice issued and moved through its status ladder, and offence history correctly queried, all testable independent of any frontend.

---

2. AI/OCR + NLP Engineer

Python + FastAPI AI service

What to Build

- **Image preprocessing** — basic OpenCV cleanup (deskew, contrast boost, crop) on each of the 2–3 uploaded angle-photos before OCR.
- **OCR extraction** — Tesseract (pytesseract) or EasyOCR on each image; merge and de-duplicate extracted text across the angles into one combined text block.
- **Field extraction** — pattern/regex rules to pull MRP, net quantity/weight, mfg date, expiry date, manufacturer name & address, consumer care details, country of origin. Build and tune this iteratively against real product photos.
- **Compliance rule engine** — load the Rulebook JSON (Member 5) at startup; for each required field, check presence and format validity; output a structured violations list.
- **Offence matching** — normalize product + manufacturer name, run rapidfuzz fuzzy matching against existing offence records to catch near-duplicates, and determine offence tier.
- **NLP Notice Generator** — take case data + the notice template placeholders (Member 5) and fill them in (Jinja2-style templating) to produce final notice text in English and one regional language.

Endpoints to expose (FastAPI):

- `POST /ocr/scan` — multipart image upload → extracted fields + violations + offence tier
- `POST /ocr/self-check` — same pipeline, flagged private, skips the offence-tier step
- `POST /notices/generate` — case data JSON → filled notice text (both languages)

Connects With

Needs from Member 5 (Rulebook) — hard blocker:
at least a draft rulebook.json and notice templates by end of Week 1 to start building the rule engine and generator at all.

Needs from Member 1:
agreement on the exact request/response JSON shape (Day 1–2), and confirmation that NestJS calls this service internally rather than the mobile app calling it directly.

Provides to Member 1:
the finalized OCR result and notice-generation JSON schemas, which shape the ocr_results and notices tables.

Provides to Member 3 (via Member 1):

the field names in the OCR response are what the Flutter review screen renders as editable fields — cross-check this schema directly once stable.

Demo Deliverable

A working FastAPI service, demoable directly via its own Swagger UI (/docs) — upload real packaging photos, get back structured violation data and a generated notice, and a working offence-tier check against a small test dataset of past offences.

---

3. Mobile App Developer (Flutter — Inspector + Business)

The Flutter app, both Inspector and Business modes

What to Build — Inspector mode

- Login screen
- Business search/select screen (GET /businesses?search=)
- New inspection screen — start an inspection against a selected business (optionally linked to a citizen complaint)
- Multi-angle camera capture screen (2–3 photos), uploads to the backend
- OCR review screen — extracted fields shown in **editable** form fields so the inspector can correct anything before proceeding
- Offence-tier display (first/second offence, shown after backend responds)
- Notice screen — review the auto-generated notice text, capture a digital signature (signature-pad widget), submit to issue
- Panchanama form — two witness fields (name + contact), seizure sample photo upload, auto-generated Sample ID
- Offline mode — local storage (sqflite/Hive) for inspection data with no signal, with a sync manager that pushes queued records once back online

What to Build — Business mode

- Registration screen (name, address, GSTIN, turnover)
- Self-check screen — same multi-angle camera flow, calls /self-check, shows results privately with a clear "Private — never shared with inspectors" label
- Notice inbox — list + detail view of notices received
- Notice detail actions: Submit Correction (re-upload photos), Raise Dispute (text box), Give Consent, Pay Penalty (Razorpay checkout)

Connects With

Needs from Member 1:
the full API contract (Postman collection) — can start building UI against mock JSON immediately, without waiting for the live backend.

Needs from Member 2 (via Member 1):
the exact OCR response field names, to build the editable review screen correctly.

Needs from Member 6:
Razorpay test keys and SDK integration snippet for the payment screen.

Provides to Member 6:
a working app to run end-to-end QA against, and screen recordings for the presentation deck.

Demo Deliverable

An installable APK (or emulator build) demonstrating the complete inspector flow (select business → scan → review → generate notice → sign) and the complete business flow (register → self-check → view private results → view a notice → dispute or pay).

SMALL TASKS

4. Web Frontend Developer (Next.js — Citizen + Controller)

Web dashboards for Citizen and Controller

What to Build — Citizen

- Signup with OTP verification
- Complaint form — category dropdown, multi-photo + invoice upload, retailer name/address fields with autocomplete against GET /businesses
- "My Complaints" list + status-tracker detail view
- Incentive status badge (Pending / Credited)

What to Build — Controller

- Login (Controller role)
- Dashboard home — stat cards and region-wise table (GET /controller/dashboard/stats)
- Case queue — pending Compounded Orders, with Approve / Reject (+comment) / Escalate to Prosecution actions
- Supply-chain assignment screen — pending supply_chain_links, assign to an inspector by jurisdiction

Connects With

Needs from Member 1:
full API docs for the endpoints above.

Needs from Member 5 (nice-to-have, not blocking):
the violation-category list — start with a hardcoded version, swap in the Rulebook-driven list once ready.

Provides to Member 1:
the Controller's Approve/Reject/Escalate actions trigger backend status transitions — confirm the exact request payload together.

Provides to Member 6:
a working web app for end-to-end testing and demo screenshots.

Demo Deliverable

A working web app — file and track a complaint as a citizen, and approve/reject/escalate a case as the Controller.

---

5. Database / Legal Knowledge Base Maker

The Legal Metrology rulebook — the most upstream role

What to Build

- Read the Legal Metrology Act, 2011 and the Legal Metrology (Packaged Commodities) Rules, 2011, focusing especially on Rule 6 (mandatory declarations).
- Build rulebook.json — one entry per mandatory field (see structure below).
- Build the deadline-days config per notice type — use the Rules' actual timelines where explicit; where the Act doesn't specify one, use a clearly-labeled configurable default rather than guessing silently.
- Draft the four notice templates (Improvement, Seizure, Panchanama, Compounded Order) with placeholders, in English and one regional language, each citing the correct Act/Rule section.
- Hand off to Member 2 and Member 1 — first draft by end of Week 1, refine through Week 2.

```
{
  "field_key": "mrp",
  "field_label": "Maximum Retail Price",
  "requirement": "Must be printed as 'MRP: Rs. XX inclusive of all taxes'",
  "act_section": "Rule 6(1)(f)",
  "penalty_first_offence": "...",
  "penalty_second_offence": "..."
}
```

Cover: MRP, net quantity, mfg date, expiry date, manufacturer name & address, consumer care detail, country of origin, unit sale price, generic name.

Connects With

Needs from anyone:
nothing — this is the one role that should start on Day 1 with zero dependencies.

Blocks:
Member 2's entire rule engine and notice generator, Member 1's penalty/deadline schema, and (loosely) Member 4's category dropdown. This is the most upstream dependency in the project — a delay here delays almost everyone else.

Demo Deliverable

rulebook.json (or a spreadsheet the team can convert), the four bilingual notice templates, and a one-page cheat sheet of the Act's core packaging requirements — genuinely useful to show judges as evidence of real domain research.

Note: this is not "small" in importance — it's smaller in coding effort, but the entire OCR compliance engine and notice generator are useless without it.

---

6. Integrations, QA & Presentation Owner

Payments, mocked integrations, testing, and the pitch

What to Build

- **Razorpay** — sandbox account, SDK integration into Member 3's payment screen, and the /payments/webhook handler built together with Member 1.
- **Mocked integrations** — GSTIN validation as a format/regex check with a hardcoded pass response; WhatsApp/Email via a free-tier service (Twilio sandbox / SendGrid) triggered at 2–3 key transitions; eMudhra stands in as the signature-pad capture already built in the Flutter app.
- **End-to-end QA** — once pieces are wired together, manually run the full path (Citizen complaint → Inspection → Notice ladder → Compounding → Controller approval → Payment) repeatedly, logging bugs to a shared tracker.
- **Presentation** — SIH pitch deck, a written demo script (exact click-by-click sequence), and a backup recorded video of the full demo.

Connects With

Needs from everyone:
a mostly-working system before full end-to-end testing can start (realistically Week 3).

Needs Razorpay setup done early (Week 1):
so Member 3 isn't blocked integrating the SDK.

Provides to everyone:
the bug list from QA — the main feedback loop back into the other five members' work during Week 4.

Demo Deliverable

A working payment flow, a resolved bug log, the final pitch deck, and a rehearsed demo script with a backup video.

INTEGRATION GUIDE — WIRING IT ALL TOGETHER

Step 1 — Agree on contracts before writing feature code (Day 1)

Before anyone builds a screen or a business-logic function, get three things written down in one shared doc, agreed by the relevant people:

- **The REST API contract** — every endpoint, method, and request/response JSON shape. Member 1 drafts it, everyone else reviews it.
- **The OCR result JSON schema** — agreed between Member 1 and Member 2.
- **The notice template placeholder format** — agreed between Member 5 and Member 2.

This single planning session prevents the most common hackathon integration failure: two people building against two different assumptions about a field name.

Step 2 — The connection map: who calls whom

Caller	Callee	Purpose

Flutter app (Inspector/Business)	NestJS backend	All mobile requests — one single base URL
Next.js app (Citizen/Controller)	NestJS backend	Same backend, different endpoints/roles
NestJS backend	FastAPI AI service	Internal call for /ocr/scan, /ocr/self-check, /notices/generate
NestJS backend	PostgreSQL	All persistent data
NestJS backend	Keycloak	Auth token validation
NestJS backend	Razorpay	Payment order creation + webhook
FastAPI AI service	rulebook.json	Loaded at startup from a shared file, updated via Member 5's commits

Key decision: the mobile and web apps only ever talk to the NestJS backend — never directly to the FastAPI service. The backend proxies AI requests internally. This gives Member 3 and Member 4 exactly one API base URL to configure, instead of two.

Step 3 — Local dev environment

Set up a docker-compose.yml with four services: postgres, keycloak, nestjs-backend, fastapi-ai-service — anyone on the team should be able to run one command and get the full backend running locally to build their frontend against. Pair this with a single shared .env.example file listing every required variable (DB connection string, Keycloak URL, Razorpay keys, AI service base URL).

Step 4 — Phased integration plan

Phase	When	What happens
1	Week 1	Everyone builds against mocks. Member 1 gives Member 3 & 4 a Postman collection with fake responses. Member 2 tests OCR standalone via FastAPI's /docs.
2	Week 2	Member 1 and Member 2 connect for real — NestJS calls the live FastAPI service for the first time. Verify one full round trip.
3	Early Wk 3	Member 3 & 4 switch from mock data to the real NestJS backend (run locally, tunneled with ngrok for a physical phone).
4	Mid Wk 3	Swap in Member 5's finalized rulebook data, replacing placeholder rules, and re-test notice generation with real legal text.
5	Late Wk 3–4	Member 6 plugs in Razorpay and notification stubs, then runs full end-to-end passes, logging bugs back to owners.
6	Week 4	Final rehearsal. Freeze the build a day or two before submission. Record the backup demo video.

Step 5 — Demo-day setup tip

You don't need real cloud deployment for an internal round. Run the backend, AI service, and Postgres via docker-compose on one laptop (or a shared free-tier cloud VM), expose it with ngrok, and point both the Flutter app and the Next.js app at that single ngrok URL for the live demo. This sidesteps DevOps work that isn't the point of the round, while still giving you a genuinely live, working system on stage.